CS 335 - Fall 2026: Assignment 2

Instructor: Leili Rafiee Sevyeri Email: `Leili.rafiee.sevyeri@uwaterloo.ca`

Due: Monday, Oct 26, 2025, 5:00pm.

Submit all components of your solutions (written/analytical work, code/scripts/notebooks, figures, plots, etc.) to Crowdmark in PDF form in the section for each question.
You must also separately submit a single zip file containing any and all code/notebooks/scripts you write to the A2 DropBox on LEARN, in runnable format (that is, .ipynb).
For full marks, you must show your work and adequately explain your logic!

1. (15 marks) Option pricing using a binomial lattice.

   Coding: Develop Jupyter/Python code for pricing European and American options, of both put and call type, using a binomial lattice. Use the lattice parameters for the no-arbitrage lattice, as described for example in Section 3.2 of the course notes (and in lectures). For this question, you do not need to apply smoothing of the payoff.

   Your code should be vectorized so that there is only one explicit for loop over the timesteps (i.e., no for loops over vector entries within one timestep). See Section 3.5 of the course notes for hints on how to proceed. Also, your code should take only $O(N)$ storage, NOT $O(N^2)$. That is, do not store data for the whole tree at once, but rather data for one or two columns at a time. In the American case, ensure that you compute the payoff at each time step using vector operations; this will require determining/knowing the vector of stock prices for that time/column.

   Test your code for European put/call options code by using the data in Table 1 below. Compare with the exact analytical solution values given by the function `blsprice` provided with the assignment. (`blsprice` is designed to mimic the Matlab function of the same name, so you can find details on it here: https://www.mathworks.com/help/finance/blsprice.html)

   Generate text tables showing the option value $V_{\Delta t}(S_0^0, 0)$ (for **both** puts and calls) as a function of Δt. Start off with 500 timesteps (i.e., $\Delta t = T/500$), and then repeat the process doubling the number of timesteps, i.e., $1000, 2000, \ldots$ 8000, timesteps. You should expect to see your results converging towards the exact solution value. Your tables be structured roughly like Table 2 (but you do not need to make fancy borders - just rows of text is fine). Your table should also show the exact (`blsprice`) value for comparison (where available).

σ	0.32
r	0.04
Time to expiry	1 year
Strike price K	\$100
Initial asset price S_0^0	\$100

Table 1: Some typical option parameters

   If V^{exact} is the exact price, and V^{tree} is the price from a lattice pricer, then it is possible to prove that

$$V_{\Delta t}^{tree}(S,t) \quad = \quad V^{exact}(S,t) + C(S,t)\Delta t + O((\Delta t)^2), \tag{1}$$

Δt	Value	Change	Ratio
T/500	$V_{\Delta t_1}$		
T/1000	$V_{\Delta t_2}$	$V_{\Delta t_2} - V_{\Delta t_1}$	
T/2000	$V_{\Delta t_3}$	$V_{\Delta t_3} - V_{\Delta t_2}$	$\frac{V_{\Delta t_2} - V_{\Delta t_1}}{V_{\Delta t_3} - V_{\Delta t_2}}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
exact	V^{exact}		

Table 2: Convergence Test

where $C(S,t)$ is independent of Δt. Estimate (by hand calculation using the above formula)

$$\lim_{\Delta t \to 0} \frac{V^{tree}_{\Delta t/2}(S^0_0, 0) - V^{tree}_{\Delta t}(S^0_0, 0)}{V^{tree}_{\Delta t/4}(S^0_0, 0) - V^{tree}_{\Delta t/2}(S^0_0, 0)} \ . \tag{2}$$

What order of accuracy does the convergence data table you generated indicate our lattice method is likely exhibiting? Does this agree with the theory above?

Next, repeat the above tests for an **American put** (no analytic solution available), using the data in Table 1. Show a convergence table as in Table 2 (but without the exact solution).

Solution:

See provided code for my solution.

Estimating the value of the expression above. I'll drop the parameters and superscript "tree" just for brevity.

$$\frac{V^{tree}_{\Delta t/2} - V^{tree}_{\Delta t}}{V^{tree}_{\Delta t/4} - V^{tree}_{\Delta t/2})} = \frac{(V^{exact} + C\Delta t/2 + O(\Delta t^2)) - (V^{exact} + C\Delta t + O(\Delta t)^2)}{(V^{exact} + C\Delta t/4 + O(\Delta t^2)) - (V^{exact} + C\Delta t/2 + O(\Delta t)^2)} \tag{3}$$

$$= \frac{C(\Delta t/2 - \Delta t) + O(\Delta t^2)}{C(\Delta t/4 - \Delta t/2) + O(\Delta t^2)} \tag{4}$$

$$= \frac{C(\Delta t/2) + O(\Delta t^2)}{C(\Delta t/4) + O(\Delta t^2)} \tag{5}$$

$$= \frac{C(\Delta t) + O(\Delta t^2)}{C(\Delta t/2) + O(\Delta t^2)} \tag{6}$$

$$= \frac{1 + O(\Delta t)}{1/2 + O(\Delta t)} \tag{7}$$

In the limit of small Δt, this becomes 2.

Similarly, based on the observed data, we see the ratio is around 2, which implies that the method is first order accurate, which is indeed consistent with equation (1) in the question. In the European cases (put and call), the table should be converging towards the analytical solutions given by the blsprice function.

2. (5 marks) Smoothing the Payoff.

In this question, we consider a variant of a put option known as a *digital put* option. Such an option is characterized by a strike price K and a payoff according to

$$\text{Payoff} = \begin{cases} 1 & \text{if } K > S, \\ 0 & \text{otherwise.} \end{cases}$$

where (as usual) S is the stock value. Determine the expressions for the *smoothed* digital put payoff in this case. You should use the same smoothing strategy as covered in the lecture (which will, of course, have to be modified for a digital put).

Solution:

Okay, so we're going to follow the same structure as in the class notes, but noting that we have a different simpler (binary) payoff instead of the usual put or call rules. Everything else remains basically the same...

We have $g(X_j^N) = 1$ if $K > S$ and $g(X_j^N) = 0$ otherwise. Following the notes, we want to find

$$\hat{g}(X_j^N) = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{X_j^N - \sigma\sqrt{\Delta t}}^{X_j^N + \sigma\sqrt{\Delta t}} g(x)dx \tag{8}$$

which is the integral of the payoff over a region around X_j^N. We do the same transformation as the notes, with $u = x - X_j^N$.

$$\hat{g}(X_j^N) = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{\sigma\sqrt{\Delta t}} g(X_j^N + u)du \tag{9}$$

So far it's all the same, since we just have a different g function. Now, we need to sub in our actual g function for the payoff, so its behavior matters... Three cases to think about: the discontinuity happens before the lower bound of the integral, or it happens after the upper bound of the integral, or it happens somewhere in the middle of the integral's range.

Case 1: If $K < S_j^N e^{-\sigma\sqrt{\Delta t}}$, we have the first situation, so the payoff is 0 for the whole range of the integral. Therefore $\hat{g}(X_j^N) = 0$.

Case 2: If $K > S_j^N e^{+\sigma\sqrt{\Delta t}}$ then we have the second situation, where the discontinuity occurs above the range, so we have 1 over this whole integral.

Intuitively it's clear that the integrated average of 1 over the range is 1, but one can also do the dirty work of verifying explicitly, if you are so inclined:

$$\hat{g}(X_j^N) = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{\sigma\sqrt{\Delta t}} 1du = \frac{1}{2\sigma\sqrt{\Delta t}}(\sigma\sqrt{\Delta t} - -\sigma\sqrt{\Delta t}) = 1 \tag{10}$$

Case 3: This is the slightly trickier case. As in the notes, we want to know where the discontinuity occurs inside the interval. This happens at the precise point where S=K, so $S_j^N e^u = K$ implies $u = \log(K/S_j^N)$.

We integrate over the non-zero part of the integral's domain:

$$\hat{g}(X_j^N) = \frac{1}{2\sigma\sqrt{\Delta t}} \int_{-\sigma\sqrt{\Delta t}}^{\log(K/S_j^N)} 1 du \tag{11}$$

$$= \frac{1}{2\sigma\sqrt{\Delta t}} (\log(K/S_j^N) + \sigma\sqrt{\Delta t}) \tag{12}$$

$$= \frac{\log(K/S_j^N) + \sigma\sqrt{\Delta t}}{2\sigma\sqrt{\Delta t}} \tag{13}$$

3. (8 marks) Lattice properties.

Let C_j^n be the value of a European *call* option at node (j, n), obtained using a no-arbitrage lattice. Likewise, let P_j^n be the value of a European *put* option at node (j, n), obtained using a no-arbitrage lattice. Assume that P_j^n and C_j^n both have the same strike price K, are based on the same underlying stock S, and have expiry time $T = N\Delta t$. Show that

$$C_j^n - P_j^n + Ke^{-(N-n)r\Delta t} = S_j^n. \tag{14}$$

This is a discrete version of what is called *put-call parity.*

Hint: Induction is often useful for understanding lattices, and that is the case here as well. Work backwards from the expiry time (i.e., column N in the lattice) and use the rules that define the relationships between S at adjacent timesteps, and likewise for the option price V at adjacent timesteps (in this case we have used C or P rather than V to distinguish the two types of options, but the update rule remains the same).

Solution:

First, let's do time N as a base case:

$$C_j^N - P_j^N = \max(S_j^N - K, 0) - \max(K - S_j^N) = S_j^N - K = S_j^N - Ke^{-(N-N)r\Delta t} \tag{15}$$

And thus we have:

$$C_j^N - P_j^N = S_j^N - Ke^{-(N-N)r\Delta t} \tag{16}$$

consistent with the given statement for the case $n = N$.

Now we will use induction, so assume the claim holds true for all entries at time/column $n + 1$, and show it holds for n.

By our update rule for option prices (i.e., discounted expected value under no-arbitrage / risk neutral probabilities), we have:

$$C_j^n - P_j^n = e^{-r\Delta t} \left(p^* C_{j+1}^{n+1} + (1 - p^*) C_j^{n+1} \right) - e^{-r\Delta t} \left(p^* P_{j+1}^{n+1} + (1 - p^*) P_j^{n+1} \right) \tag{17}$$

$$= e^{-r\Delta t} \left(p^* (C_{j+1}^{n+1} - P_{j+1}^{n+1}) + (1 - p^*)(C_j^{n+1} - P_j^{n+1}) \right) \tag{18}$$

Now we exploit the fact that it's true for $n + 1$ to get:

$$C_j^n - P_j^n = e^{-r\Delta t} \left(p^* (S_{j+1}^{n+1} - Ke^{-(N-n-1)r\Delta t}) + (1 - p^*)(S_j^{n+1} - Ke^{-(N-n-1)r\Delta t}) \right) \tag{19}$$

The K's can be pulled out, using that $p^* + (1 - p^*) = 1$:

$$C_j^n - P_j^n = -Ke^{-(N-n)r\Delta t} + e^{-r\Delta t}\left(p^*(S_{j+1}^{n+1}) + (1-p^*)(S_j^{n+1})\right) \tag{20}$$

Now let's use the update rule for S:

$$C_j^n - P_j^n = -Ke^{-(N-n)r\Delta t} + e^{-r\Delta t}\left(p^* e^{\sigma\sqrt{\Delta t}} S_j^n + (1-p^*) S_j^n e^{-\sigma\sqrt{\Delta t}}\right) \tag{21}$$

$$= -Ke^{-(N-n)r\Delta t} + e^{-r\Delta t} S_j^n \left(p^* e^{\sigma\sqrt{\Delta t}} + (1-p^*) e^{-\sigma\sqrt{\Delta t}}\right) \tag{22}$$

Now we just have to plug in the defs of p^* and simplify.

$$C_j^n - P_j^n = -Ke^{-(N-n)r\Delta t} + e^{-r\Delta t} S_j^n \left(\frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} e^{\sigma\sqrt{\Delta t}} + \frac{-e^{r\Delta t} + e^{\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} e^{-\sigma\sqrt{\Delta t}}\right) \tag{23}$$

$$= -Ke^{-(N-n)r\Delta t} + \frac{e^{-r\Delta t}}{e^{-\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} S_j^n \left((e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}) e^{\sigma\sqrt{\Delta t}} + (-e^{r\Delta t} + e^{\sigma\sqrt{\Delta t}}) e^{-\sigma\sqrt{\Delta t}}\right) \tag{24}$$

$$= -Ke^{-(N-n)r\Delta t} + \frac{e^{-r\Delta t}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} S_j^n \left(e^{r\Delta t} e^{\sigma\sqrt{\Delta t}} - e^0 - e^{r\Delta t} e^{-\sigma\sqrt{\Delta t}} + e^0\right) \tag{25}$$

$$= -Ke^{-(N-n)r\Delta t} + \frac{e^{-r\Delta t}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}} e^{r\Delta t} S_j^n \left(e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}\right) \tag{26}$$

$$= -Ke^{-(N-n)r\Delta t} + S_j^n \tag{27}$$

Thus we have shown $C_j^n - P_j^n + Ke^{-(N-n)r\Delta t} = S_j^n$ for the case n, as desired.

Since it holds for n given $n+1$ and it also holds for the base case N, it must hold for all values of n down to $n = 0$ (beyond which point we don't have any more update rules or lattice points).

4. (8 marks) Dynamic Programming.

   Consider the following game. You are given a fair **five-sided** die, with faces $\{0, 1, 2, 3, 4\}$. You can roll the die, and obtain a payoff equal to the value shown on the die, or you can choose to roll again. Note that if you choose to roll again, you can only get the new payoff or roll again. You can roll the die up to three times. You must pay \$1.4 for the first roll, \$1.05 for a second roll, and \$0.85. for a third roll. You pay only for the rolls you choose to take, and get no money back if you move onto the next roll. What is your expected gain, assuming you were to behave optimally at each step? (Hint: this is somewhat similar to the marriage problem we looked at in lecture. Use dynamic programming; i.e., work backwards from the last roll!)

   Work this out manually, i.e., you do not need to write or run any code here.

   Solution: We follow generally the same approach as in the "marriage problem" in the class/course notes.

   First, the possible values earned from rolling the die, considering the cost of doing the rolls is:

   - Roll 1: {-1.4, -0.4, 0.6, 1.6, 2.6}
   - Roll 2: {-2.45,-1.45,-0.45,0.55,1.55}

- Roll 3: {-3.3,-2.3,-1.3,-0.3, 0.7}

The expected value of the final roll is thus $V_3 = (-3.3 - 2.3 - 1.3 - 0.3 + 0.7)/5 = -1.3$.

Considering the possible values on roll 2, the probability that the expected value of roll 2 is better than -1.3 is 3/5, since 3 of the 5 possible rolls have higher value. Hence $p_2 = p(W_2 > V3) = p(W_2 > -1.3) = 3/5$.

The expected value $\overline{W}_2$ given that we stick with roll 2's result is then $(-0.45 + 0.55 + 1.55)/3 = 0.55$.

So our expected payout, acting optimally, from roll 2 onward is

$$V_2 = p_2\overline{W}_2 + (1 - p_2)V_3 = (3/5)(0.55) + (2/5)(-1.3) = -0.19$$

Repeat the process for the first roll:

$$p_1 = p(W_1 > -0.19) = 3/5$$

$$\overline{W}_1 = (0.60 + 1.60 + 2.60)/3 = 1.6$$

$$V_1 = (3/5)(1.6) + (2/5)(-0.19) = 0.96 - 0.076 = 0.884.$$

So our expected winnings is \$0.884, i.e., about 88 cents, assuming we play optimally.